Володимирський педагогічний фаховий коледж імені Агатангела Кримського
циклова комісія викладачів професійної цифрової та математичної освіти

Інструкція до практичної роботи №43

ТЕМА: (Нова) Надійна обробка помилок при роботі з файлами та некоректним введенням даних.

Освітньо-професійна програма: Професійна освіта (Цифрові технології)
Тривалість заняття: 80 хв

Мета заняття:

- закріпити навички обробки виняткових ситуацій на прикладі роботи з файлами;
- навчитися коректно обробляти помилки, пов'язані з некоректним введенням даних;
- ознайомитися з типовими винятками при роботі з файлами (FileNotFoundError) та даними (KeyError);
- розвинути навички написання надійних, «стійких до збоїв» програм.

Теоретичні відомості

1. Типові помилки при роботі з файлами

Робота з файлами пов'язана з низкою ситуацій, які програма не може повністю контролювати: файл може бути відсутнім, пошкодженим, або в іншому форматі, ніж очікується. Найпоширеніший виняток при роботі з файлами — FileNotFoundError, що виникає при спробі відкрити для читання файл, якого не існує.

```
try:
    with open('data.txt', 'r', encoding='utf-8') as f:
        content = f.read()
        print(content)
except FileNotFoundError:
    print('Файл не знайдено. Перевірте назву файлу.')
```

2. Обробка некоректного вмісту файлу

Навіть якщо файл існує, дані в ньому можуть виявитися не в тому форматі, який очікує програма (наприклад, замість числа — текст). У такому разі варто перехоплювати ValueError під час перетворення даних:

```
try:
    with open('numbers.txt', 'r', encoding='utf-8') as f:
        total = 0
        for line in f:
```

```
            total = total + int(line.strip())
    print('Сума чисел:', total)
except FileNotFoundError:
    print('Файл numbers.txt не знайдено')
except ValueError:
    print('Файл містить рядок, який не є цілим числом')
```

3. Обробка помилок при роботі зі словниками (JSON-даними)

Під час роботи зі структурованими даними (наприклад, зчитаними з JSON) поширена помилка — звернення до ключа, якого немає в словнику (KeyError). Її можна або перехопити через try...except, або уникнути одразу, скориставшись методом get():

```
student = {"ім'я": 'Іван', 'вік': 19}
try:
    print(student['місто'])
except KeyError:
    print('У записі немає інформації про місто')
```

4. Комбінована надійна обробка: введення + файл

У реальних програмах часто потрібно одночасно убезпечитися і від некоректного введення користувача, і від проблем з файлом. Розглянемо програму, яка зберігає введене число в файл, обробляючи обидва типи можливих помилок:

```
try:
    number = int(input('Введіть число для збереження: '))
    with open('result.txt', 'w', encoding='utf-8') as f:
        f.write(str(number))
    print('Число успішно збережено')
except ValueError:
    print('Помилка: потрібно ввести ціле число')
except OSError:
    print('Помилка: не вдалося записати файл')
finally:
    print('Спробу збереження завершено')
```

OSError — це загальніший тип помилки, що охоплює різноманітні проблеми операційної системи при роботі з файлами (наприклад, відсутність прав доступу чи заповнений диск).

5. Перевірка даних до їх використання (захисне програмування)

Іноді доцільніше не чекати винятку, а заздалегідь перевірити коректність даних умовним оператором — такий підхід називають захисним програмуванням. Обидва підходи (try...except і попередня перевірка) можна поєднувати для максимальної надійності:

```
import os
filename = 'data.txt'
if os.path.exists(filename):
    with open(filename, 'r', encoding='utf-8') as f:
        print(f.read())
else:
    print('Файл не існує, пропускаю зчитування')
```

Хід роботи

Варіант і вимоги до здачі

а) N — ваш порядковий номер у списку групи (журнал). Дані завдань 1 і 2 залежать від N:

Завдання	Дані варіанта	Приклад N = 7
1	ім'я файлу missing_N.txt	missing_7.txt
2	рядки файлу: N, 12, abc, 5, N + 3	7, 12, abc, 5, 10

б) Кожне завдання — окремий файл Прізвище_ПР43_зN.py (наприклад, Ivanenko_PR43_z3.py), код текстом, а не фото чи скріншотом. Файли з прикладів із теорії та додаткові файли, яких немає в умові, здавати не потрібно.

в) Запускайте програми в IDLE або іншому середовищі на своєму комп'ютері; до звіту (документ Word) додайте скріншот виконання кожної програми.

г) Обробляйте конкретні винятки (FileNotFoundError, ValueError, KeyError, OSError), а не голе except. Файли відкривайте через with open(..., encoding='utf-8'). Додаткові файли (numbers.txt, student.json тощо) здавайте разом із програмою.

д) У коді залиште короткі коментарі власними словами перед основними блоками.

е) Будьте готові пояснити будь-який рядок свого коду й внести в нього невелику зміну (див. розділ «Захист роботи»).

1. Створіть файл Прізвище_ПР43_з1.py. Спробуйте відкрити для читання файл missing_N.txt (N — ваш номер), якого не існує, і обробіть FileNotFoundError: «Помилка: файл missing_N.txt не знайдено».

   Приклад (N = 7): «Помилка: файл missing_7.txt не знайдено» (програма завершується без трасування помилки).

2. Створіть файл numbers.txt (по числу в рядку): N, 12, abc, 5, N + 3. Обчисліть суму чисел, обробивши ValueError: для некоректного рядка виведіть «Помилка у рядку k: 'текст' — не число» і продовжіть; наприкінці «Сума: …».

   Приклад (N = 7): рядки 7, 12, abc, 5, 10 → «Помилка у рядку 3: 'abc' — не число», «Сума: 34».

3. Зчитайте JSON-файл student.json і виведіть «Місто: …»; якщо ключа «місто» немає — «Місто: немає даних» (обробіть KeyError). Перевірте на двох файлах: з містом і без.

   Приклад: {"ім'я": "Олег", "місто": "Львів"} → «Місто: Львів»; {"ім'я": "Іра"} → «Місто: немає даних».

4. Запитайте ім'я файлу й число та запишіть число у файл. Помилку введення (ValueError) обробіть повідомленням «Помилка: введіть число», помилку запису (OSError) — «Помилка запису файлу»; успіх — «Збережено».

   Приклад: result.txt і 42.5 → «Збережено» (у файлі 42.5); abc → «Помилка: введіть число»; ім'я a/b/c.txt (папок немає) → «Помилка запису файлу».

5. За допомогою os.path.exists() перевірте наявність config.json перед читанням: якщо файлу немає — «Файл config.json не знайдено», інакше виведіть його вміст («Налаштування: …»).

   Приклад: файлу немає → «Файл config.json не знайдено»; файл {"тема": "dark"} → «Налаштування: {'тема': 'dark'}».

6. Завдання підвищеної складності*. «Журнал відвідування»: прочитайте students.json (список записів {"name", "age"}; якщо файлу немає — порожній список), додайте студента з введенням віку в циклі, доки вік коректний (число від 14 до 100; інакше «Помилка: вік має бути числом» або «Помилка: вік має бути від 14 до 100»), збережіть список назад і виведіть «Усього студентів: …».

Приклад: файлу немає, додано Олег, вік «abc» → помилка, потім 17 → «Усього студентів: 1»; після другого запуску з Іра, 18 → «Усього студентів: 2».

Критерії оцінювання

Складова	Оцінювання	Макс. балів
Завдання 1–5	кожне по 1,8 бала	9
Завдання 6*	2 бали	2
Контрольні питання	відповіді у звіті	1
Разом		**12**

Кожне завдання оцінюється так: програму виконано повністю, вона працює на вимогах умови й прикладах формату виводу — 100% балів; програма працює, але є суттєве зауваження (наприклад, голе except, програма падає на відсутньому файлі, пропущено перевірку діапазону віку, не оброблено помилку запису) — 50%; програма не працює або не здана — 0. Завдання 1–5 оцінюються по 1,8 бала, завдання 6* — 2 бали. Контрольні питання оцінюються в 1 бал, якщо на всі питання дано правильні відповіді.

Захист роботи

Перед виставленням оцінки викладач може попросити вас пояснити будь-який рядок свого коду, показати, як змінюється значення змінних на прикладі, або внести в програму невелику зміну (наприклад, додати обробку ще одного типу помилки чи змінити повідомлення). Якщо пояснити код не вдається, оцінку за відповідне завдання може бути знижено.

Контрольні питання

Відповідайте на питання письмово у звіті (документ Word разом із скріншотами): 1–2 речення на кожне питання, власними словами.

1. Яка виняткова ситуація виникає при спробі відкрити для читання неіснуючий файл?
2. Як обробити ситуацію, коли рядок з файлу неможливо перетворити на число?
3. Що таке KeyError і коли він виникає при роботі зі словниками?
4. У чому різниця між обробкою помилки через try...except і попередньою перевіркою умовним оператором (захисне програмування)?

5. Наведіть приклад ситуації, де варто поєднати обидва підходи (try...except і попередню перевірку) для максимальної надійності.